

A Data-Driven Reduced-Order Model for Friction-Stir Welding

POD/SVD Feasibility: Density Fields and Particle Clouds

FSW-ROM study (JAXTrace)

July 3, 2026

Abstract

We assess whether a reduced-order model (ROM) can predict the full 3D outcome of a friction-stir-welding (FSW) simulation from two process inputs — advance velocity v_{adv} (`INLET_VELOCITY`) and pin rotation ω_{pin} (`PIN_RPM`) — using linear Proper Orthogonal Decomposition (POD) via the SVD. Two output targets are studied: the Eulerian mean-density field and the Lagrangian particle cloud. The main conclusion is that *linear POD is feasible but sample-limited*: with 20 cases the density surrogate reaches $\sim 28\%$ held-out error, and the dominant error source is the two-input $\rightarrow$ mode-coefficient regression, not the POD basis. Two representation choices give clear gains: the *time-averaged* density is far more predictable than a single final snapshot, and a *2D* (y, z) *cross-section* that collapses the advection axis reaches the best result of the study ($\sim 21\%$). Active-subspace and manifold diagnostics show the response is v_{adv} -dominated and lives on a smooth but curved low-dimensional manifold, so the practical next step is more samples in the sparsely covered high- $|\omega_{\text{pin}}|$ /low- v_{adv} corner.

Contents

1	Dataset	2
2	Method: pre-SVD preparation, SVD, and validation	2
3	Core results	3
3.1	Density field	3
3.2	Particle cloud	4
4	Why the error is what it is: three diagnostics	6
4.1	Input features and active subspace	6
4.2	Manifold geometry	6
4.3	The bottleneck	8
5	Data-quality fixes (all 20 cases)	9
6	Representation studies	9
6.1	Final-step vs. union (time-averaged) density	9
6.2	2D (y, z) cross-section (collapse the advection axis)	9
7	Summary of variants	12

1 Dataset

Twenty full-order FSW simulations on a Latin-hypercube-like sampling of $(v_{\text{adv}}, \omega_{\text{pin}})$, with $v_{\text{adv}} \in [5.0, 10.0] \times 10^{-3}$ m/s and $\omega_{\text{pin}} \in [-800, -400]$ RPM. Authoritative parameters come from each case’s `run_jaxtrace.sh` (the CSV `dt` column is inconsistent and was not used). A notable design property: $v_{\text{adv}} \cdot t_{\text{max}} = v_{\text{adv}} \cdot \text{DT} \cdot N_{\text{STEPS}} = 0.075$ for *every* case — the runs are length-matched, so all cases advect the same total distance.

	density study	particle study
snapshot	one static 3D <code>mean_density</code> field	final-step displacement of 360k particles
cases used	20	20
source	<code>particles_union_density.vtkhdf</code>	<code>particles.vtkhdf</code>

Table 1: The two ROM targets. Both use all 20 cases after data-quality fixes (Sec. 5).

2 Method: pre-SVD preparation, SVD, and validation

Correspondence (pre-SVD). POD needs every snapshot as a vector on *one shared grid*, so that “column j ” means the same thing in every case. The native density grids differ per case (extent/spacing scale with v_{adv}), so every case is trilinearly resampled onto a common intersection-box grid, then a boundary margin is trimmed (1 voxel per wall, 6 off z_{max} to remove an artificial top-layer). For particles, the seeds are byte-identical across cases (same 360k particles, same `ParticleID` order), so correspondence is free: a per-particle displacement stacks directly.

SVD (method of snapshots). The snapshot matrix $X \in \mathbb{R}^{n \times m}$ is short and fat ($n = 20$ cases, $m \sim 10^6$ voxels). We mean-centre and factor the small Gram matrix $G = X_c X_c^\top \in \mathbb{R}^{n \times n}$ with `jax.numpy.linalg.svd`, recovering the spatial modes; this is identical to the economy SVD of X_c but avoids GPU `gesvd` failures on the wide matrix. The per-voxel mean is the POD reference field (“mode 0”); optional normalizations (global scalar, per-case L^2/mass , log) were tested but change little (Sec. 3).

Validation: projection vs. LOOCV. Two error numbers appear throughout and must not be conflated. *Projection error* fits POD on all cases and reconstructs case i from its *own* coefficients — it measures only basis expressiveness ($\sim 12\%$ at $K=7$ for density). *Leave-one-out cross-validation* (LOOCV) is the real generalization test: for each case i , fit POD on the other $n-1$, fit a regressor $(v_{\text{adv}}, \omega_{\text{pin}}) \mapsto$ mode coefficients on them, predict the held-out coefficients, reconstruct, and measure relative L^2 error:

$$\hat{\rho}_i = \bar{\rho} + \sum_{k=1}^K \hat{A}_{ik} \phi_k, \quad e_i = \frac{\|\hat{\rho}_i - \rho_i\|}{\|\rho_i\|}.$$

A surrogate cannot project an unseen field, so LOOCV necessarily tests *POD + regression together*. The gap between projection ($\sim 12\%$) and LOOCV ($\sim 28\%$) is the regression error. Three lightweight regressors are compared: `rbf` (thin-plate spline), `gp` (small NumPy Gaussian process), `poly` (degree-2).

3 Core results

3.1 Density field

The singular-value spectrum decays gently (no sharp elbow): 90% energy needs ~ 10 modes. Normalization barely moves the held-out error. LOOCV is $\sim 28\%$ (rbf); the error concentrates in the high- $|\omega_{\text{pin}}|/\text{low-}v_{\text{adv}}$ corner and at the sampling convex-hull edges. Restricting to the near-pin first 20% of x (dropping the v_{adv} -sensitive wake) improves the surrogate.

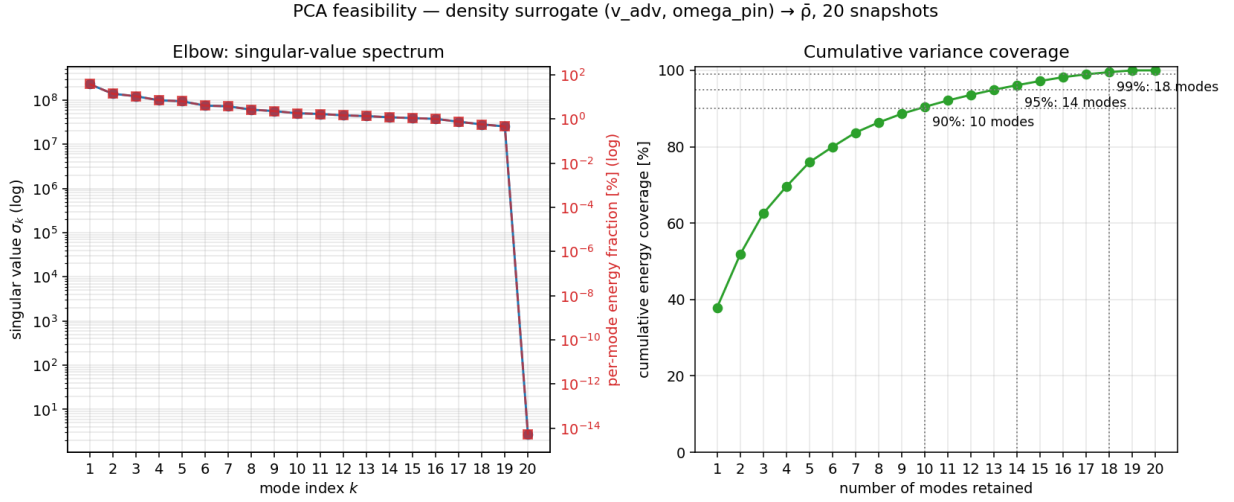


Figure 1: Density POD spectrum. Left: singular-value elbow (mode 17 is the null mode from mean-centring). Right: cumulative energy coverage, with 90/95/99% thresholds annotated.

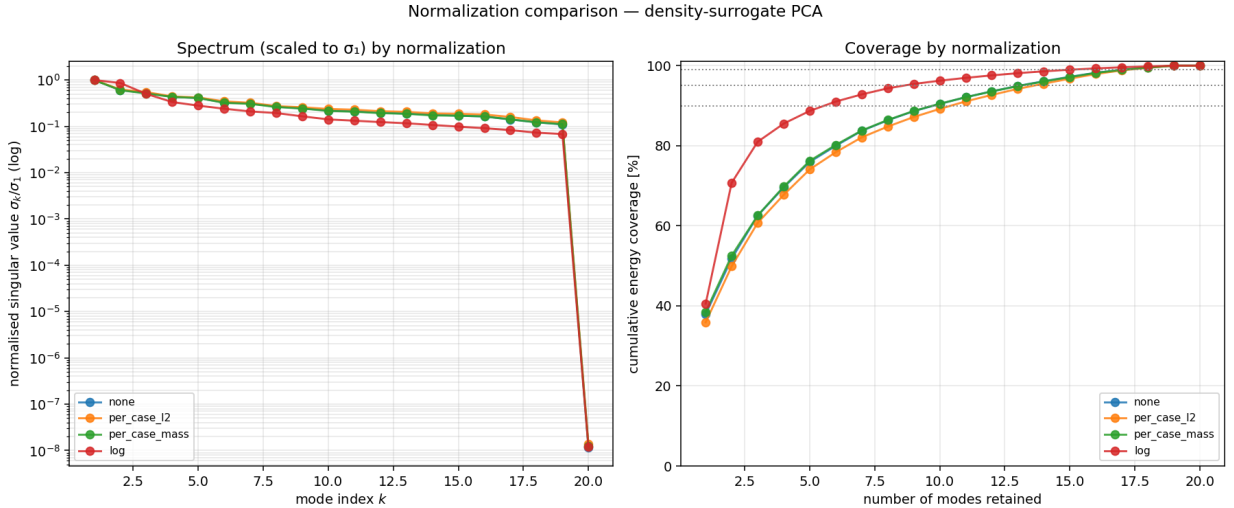


Figure 2: Normalization comparison. **per_case** options barely change the spectrum; **log** reaches coverage in fewer modes but its error is measured in log-density space and is not comparable.

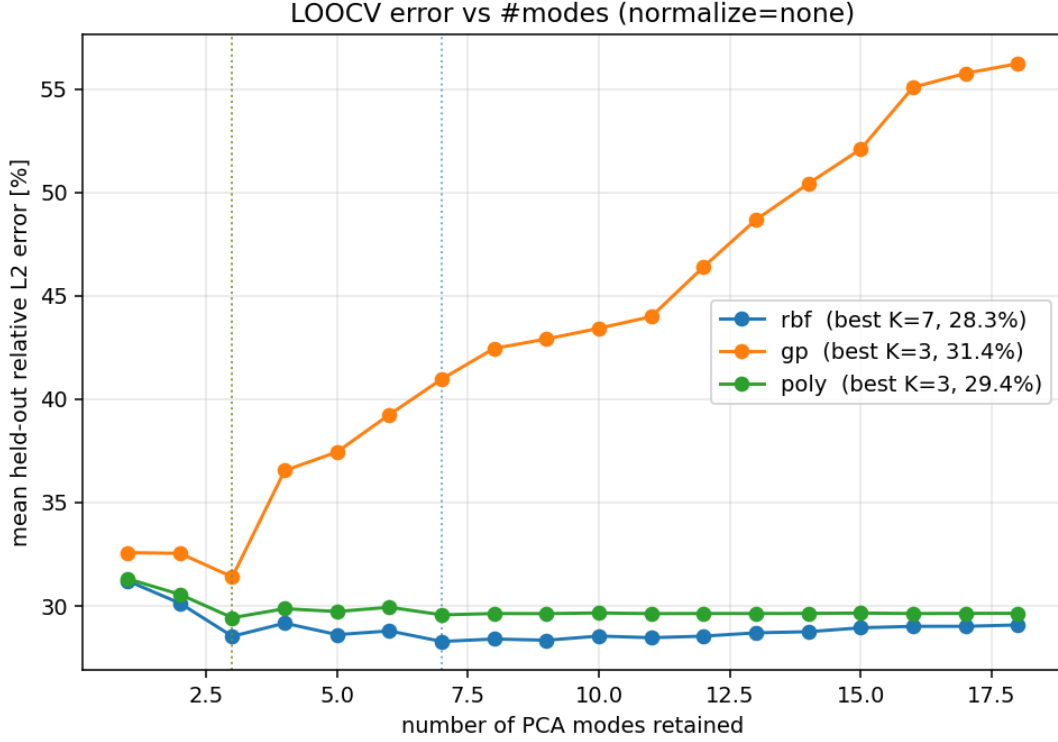


Figure 3: Density LOOCV error vs. number of retained modes, per regressor. The dip (bias/variance sweet spot) is at $K \approx 3-7$; all reasonable regressors converge to a similar floor set by the sample count.

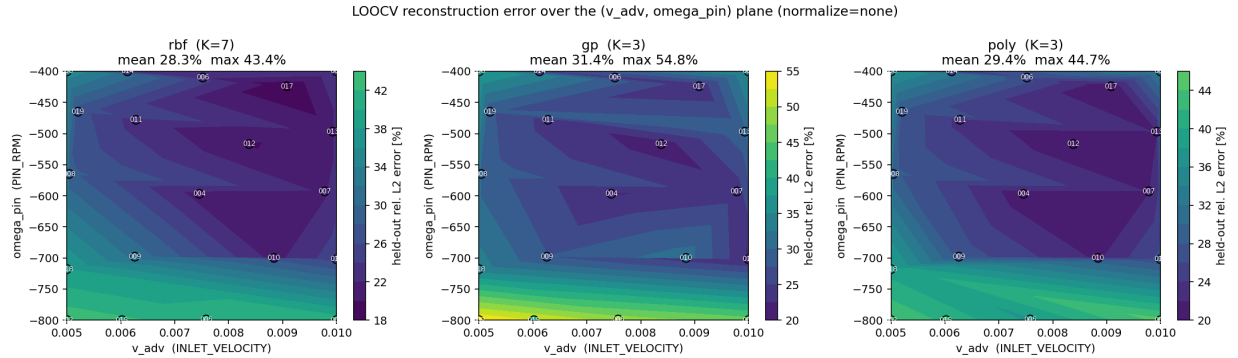


Figure 4: Density held-out error over the $(v_{\text{adv}}, \omega_{\text{pin}})$ plane. Error peaks in the high- $|\omega_{\text{pin}}|$ /low- v_{adv} corner — the sparsely sampled region.

3.2 Particle cloud

Seeds are byte-identical across cases, so the natural target is the per-particle final-step displacement. A subtle finding: because correspondence is exact, subtracting the (constant) bulk drift is a rigid offset fully absorbed by mean-centring, so raw / co-moving / final representations give *identical* POD modes. Thresholding to “eliminate unaffected particles” is moot: $> 97\%$ of particles exceed even $|\xi| > 5\Delta p$, since the seed spacing is tiny relative to typical displacement.

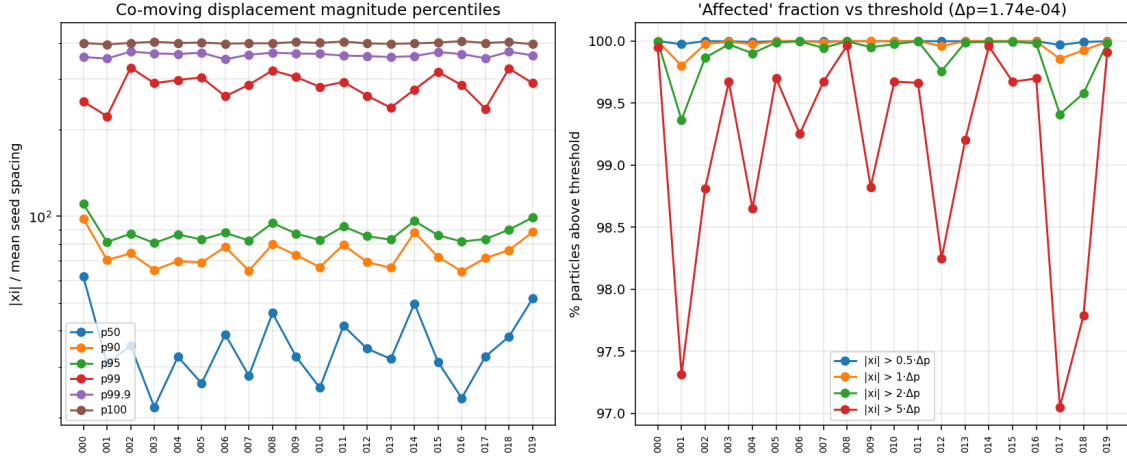


Figure 5: Co-moving displacement magnitude $|\xi|/\Delta p$ per case (left) and the “affected” fraction vs. threshold (right). Almost all particles are “affected”, so a magnitude threshold cannot isolate the stir zone.

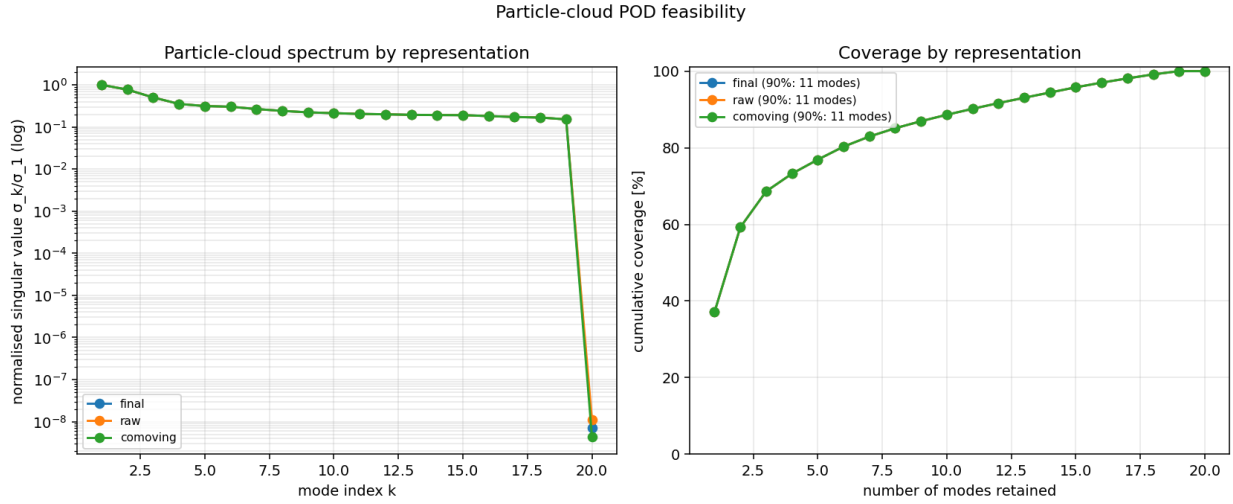


Figure 6: Particle-cloud POD by representation. The **final**, **raw**, and **comoving** curves coincide exactly — drift subtraction does not change the modes.

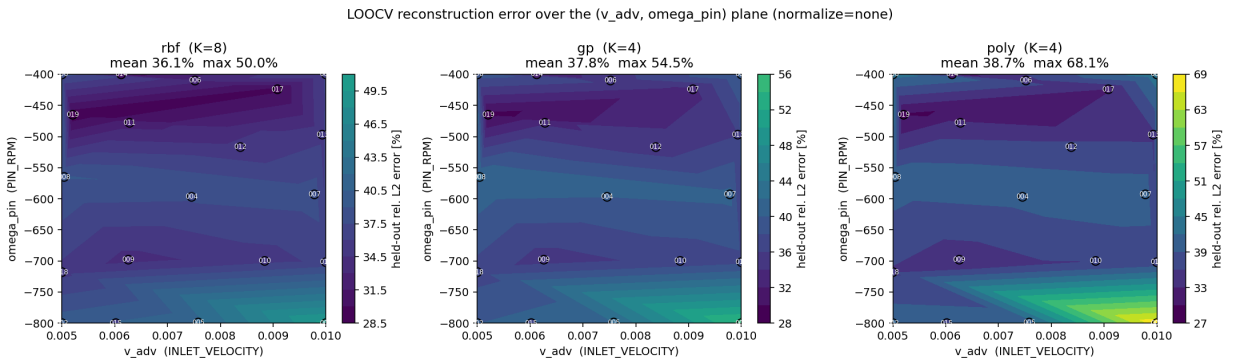


Figure 7: Particle displacement LOOCV error over the (v_{adv}, ω_{pin}) plane.

The Lagrangian displacement is harder to predict than the Eulerian density ($\sim 36\%$ vs. $\sim 28\%$ LOOCV), consistent with the literature: a density field spatially averages many trajectories and is a lower-variance, more predictable quantity.

4 Why the error is what it is: three diagnostics

4.1 Input features and active subspace

Mapping the inputs to physics-motivated features — weld pitch $p_w = v_{\text{adv}}/|\omega_{\text{pin}}|$, log inputs, and the first-stage ROM (velocity / temperature / pressure) coefficients — *none* beat the raw two-scalar baseline. An active-subspace analysis explains why: the input $\rightarrow$ coefficient map is nearly 1D, but its dominant direction is essentially *pure* v_{adv} , not weld pitch, so collapsing onto p_w is the wrong reduction. The first-stage coefficients are largely a reparametrisation of the two scalars (leading modes have $R^2 \approx 1.0$ with the inputs; only mode ≥ 3 carries new information, which does not drive the density modes), and adding all of them overfits.

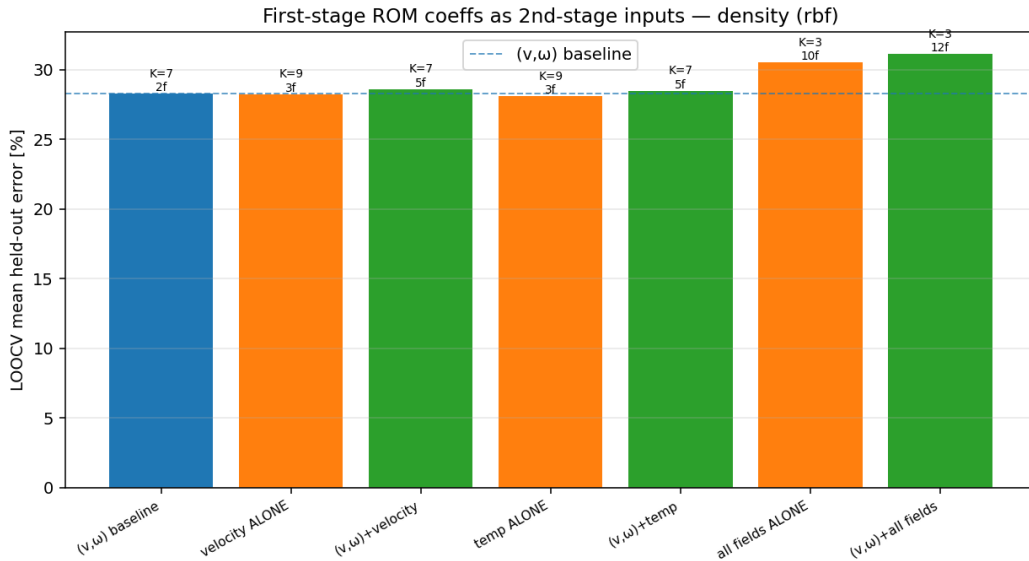


Figure 8: First-stage ROM coefficients as second-stage inputs. Velocity coefficients alone are marginal; piling on all fields overfits (12 features, 20 points $\Rightarrow$ best- K collapses to 3).

4.2 Manifold geometry

Characterising the case manifold with linear/metric methods (before any nonlinear embedding) reveals a *high linear dimension but low intrinsic dimension* — the signature of a smooth but *curved* manifold. Classical MDS axes are the variance-ordered principal directions of the case cloud; we measure their alignment with the inputs afterwards.

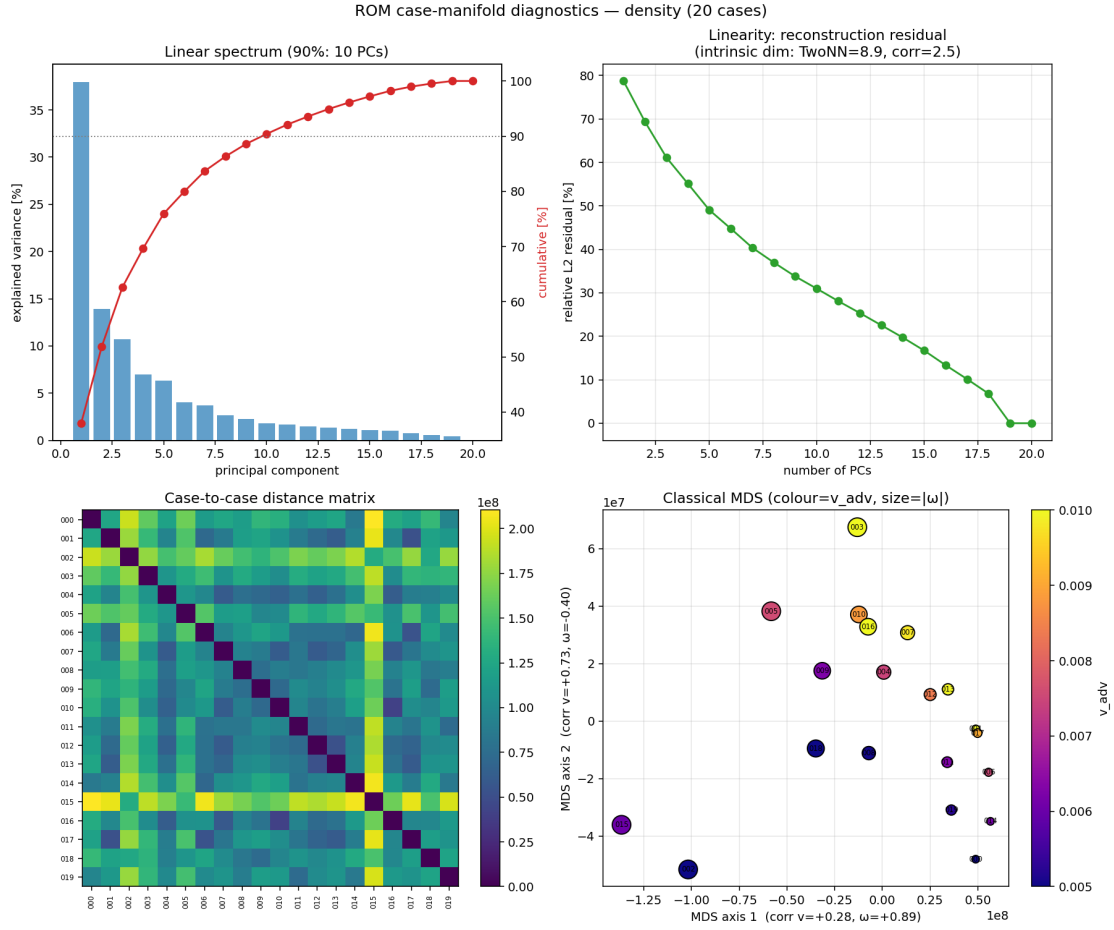


Figure 9: Density case-manifold diagnostics: spectrum, linearity residual, distance matrix, and classical-MDS layout coloured by v_{adv} / sized by $|\omega_{pin}|$. Intrinsic dim ≈ 2.5 vs. 10 linear PCs for 90%.

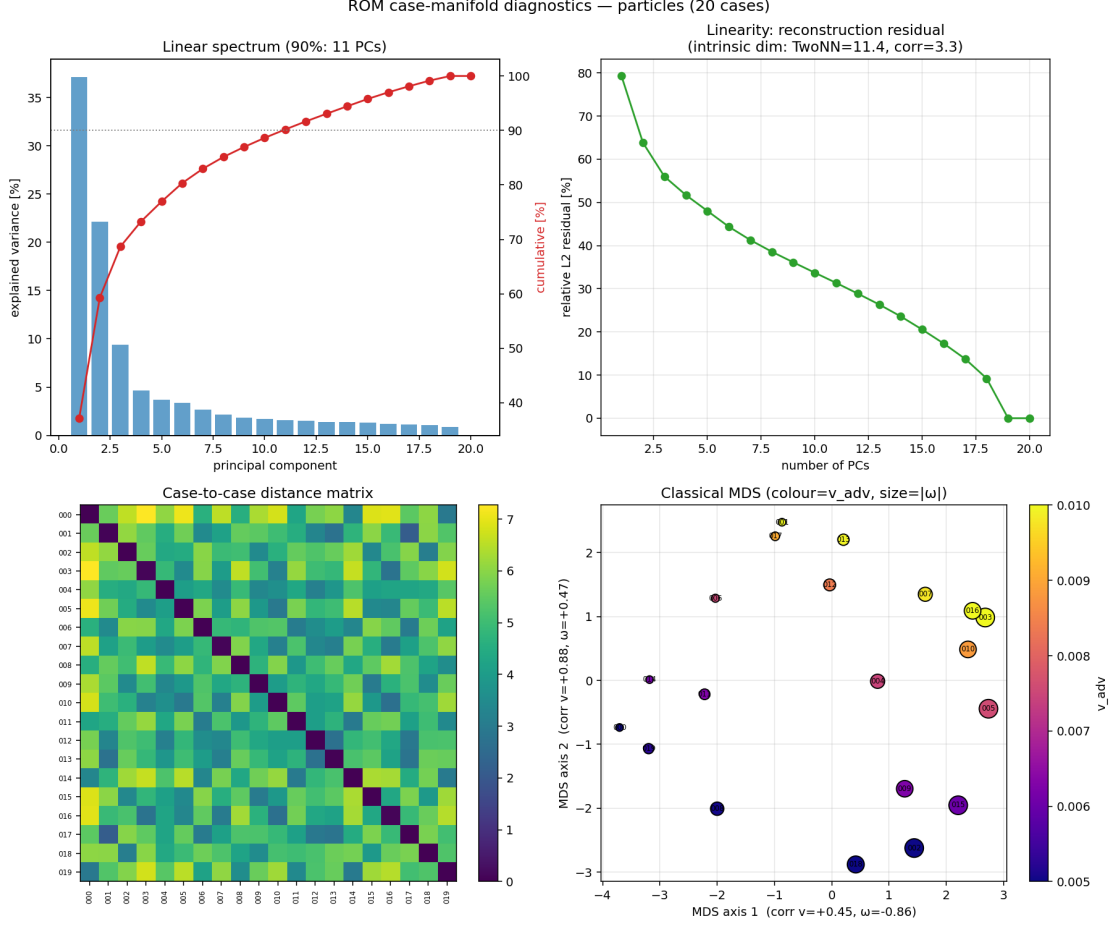


Figure 10: Particle case-manifold diagnostics. MDS axis 1 tracks ω_{pin} strongly; particles give a clean (ω_{pin}, v_{adv}) split, whereas density’s axis 2 mixes the parameters (the wake couples them).

How the MDS axes arise, and why they differ. Classical MDS double-centres the distance matrix, $B = -\frac{1}{2}JD^2J$, and takes coordinates $x_i = \sqrt{\lambda_i}v_i$ for the two largest eigenvalues — so the axes are variance-ordered and data-driven (for Euclidean D this is PCA of the cases), *not* parameter-picked. Both targets have **axis 1** $\approx \omega_{pin}$ (the largest source of case-to-case variation). Particles split cleanly (axis 2 $\approx$ pure v_{adv}); density’s axis 2 *mixes* v_{adv} and ω_{pin} because the field couples them through the wake (length $\propto v_{adv}$, pattern $\propto \omega_{pin}$). A small negative MDS eigenvalue is the expected mild-curvature signal.

4.3 The bottleneck

Projection error ($\sim 12\%$) $\ll$ LOOCV ($\sim 28\%$); neither normalization, input features, nor regressor choice moves the needle; and the active subspace is nearly 1D in v_{adv} . The limiting factor is unambiguously **sample count and placement** — 20 points in a 2D space that is effectively 1D, sparsest exactly where the error peaks.

5 Data-quality fixes (all 20 cases)

Three cases were initially set aside and later fixed, after which the entire study runs on all 20. Cases 000/001 produced $\sim 1.4\text{--}2.5\%$ *runaway particles* (flying to $x \approx 2\text{--}3.7$ m vs. a ~ 0.08 m domain); the density driver sized its voxel grid from the trajectory bbox, which those runaways inflated to $10^{11}\text{--}10^{12}$ voxels $\Rightarrow$ GPU out-of-memory. A clean rerun (plus an absolute physical ROI box) removed them. Case 002’s density had been *host*-RAM OOM-killed (batch dedup over 8001 steps $\times$ 360k particles ≈ 2.9 billion points > 58 GiB); recomputing with a step stride fixed it. Including the (now clean) cases barely moved any number, confirming the earlier exclusions did not bias the study.

6 Representation studies

6.1 Final-step vs. union (time-averaged) density

The core density study uses the *union* density (time-average over the whole trajectory). Computing instead a *final-step* density (one snapshot) and re-running the suite shows the union is markedly better for ROM (Table 2). Temporal averaging denoises the case-to-case stochastic variability of any single instant, so the union field varies more smoothly and lower-dimensionally with the inputs.

metric	union (time-avg)	final-step
PCA modes for 90%	10	12
PC-1 energy	37.9%	25.5%
LOOCV, full domain (rbf)	28.3%	49.3%
LOOCV, near-pin (rbf)	24.6%	75.2%
manifold intrinsic dim	2.5	4.8

Table 2: Union density is substantially more predictable than a single final snapshot.

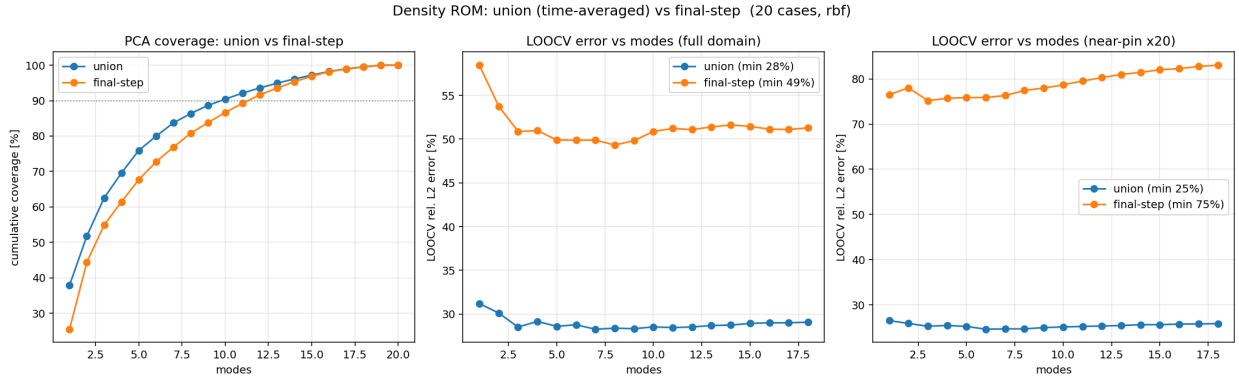


Figure 11: Union vs. final-step density: coverage curves (left) and LOOCV error vs. modes for the full domain (centre) and near-pin (right).

6.2 2D (y, z) cross-section (collapse the advection axis)

The wake stretches along x and its *length* scales with v_{adv} — the very feature that dominates the error. Collapsing x (summing the slab into a (y, z) cross-section, a $\sim 300\times$ reduction) tests whether

integrating out that axis helps. It **helps density and hurts particles**, for the same underlying reason (Table 3).

	3D	2D (y, z)
density — PCA modes for 90%	10	6
density — LOOCV (rbf)	28.3%	20.6%
density — intrinsic dim	2.5	1.7
particles — PCA modes for 90%	11	14
particles — LOOCV (rbf)	36.1%	41.4%
particles — intrinsic dim	3.3	4.3

Table 3: 2D cross-section: the best density result of the study, but worse for particles.

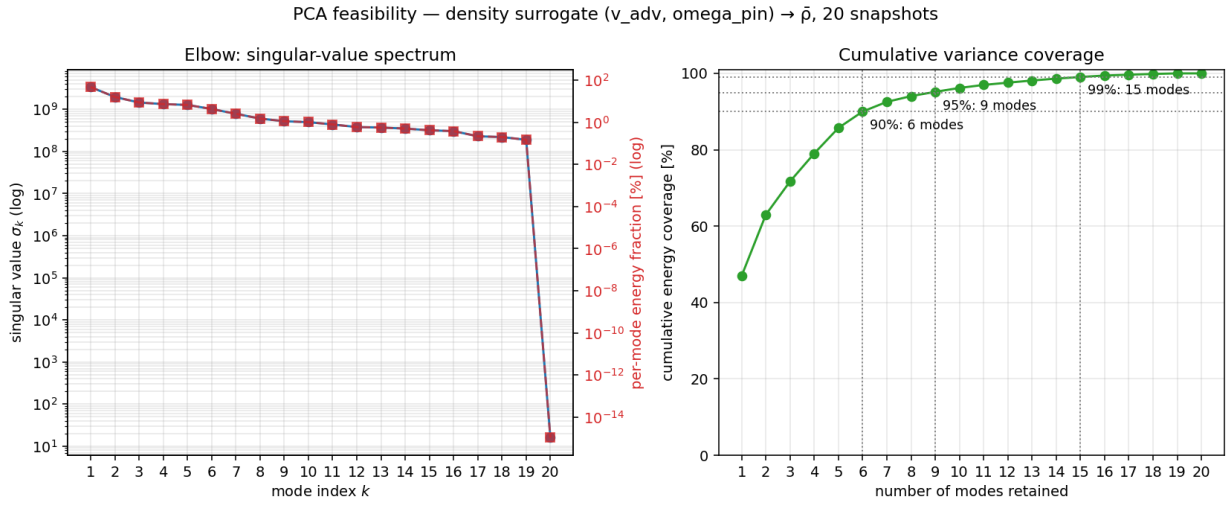


Figure 12: 2D density POD spectrum — markedly faster decay than 3D (90% in 6 modes).

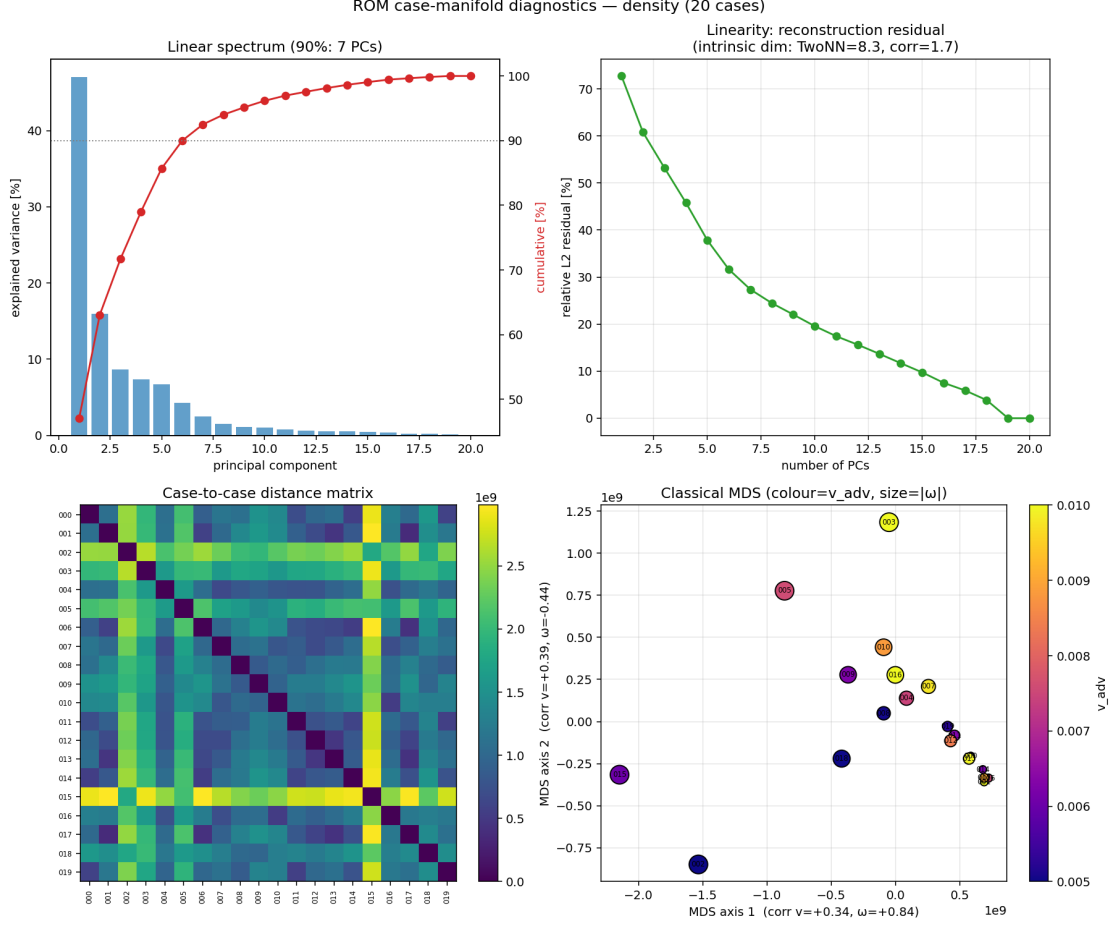


Figure 13: 2D density case-manifold: intrinsic dim ≈ 1.7 , a cleaner and lower-dimensional manifold than 3D.

For **density**, integrating along x averages out the v_{adv} -driven wake variability and per-voxel noise (the spatial analogue of the union’s temporal denoising). For **particles**, x is the large, smooth, easily-predicted advection displacement; dropping it removes the “free” mode and leaves only the small, chaotic in-plane stir motion, which is harder to predict. So for the cloud, x is signal, not noise.

7 Summary of variants

target	variant	90% modes	best LOOCV	note
density	union, full domain	10	28.3%	baseline (all 20)
density	union, near-pin x	11	24.4%	drops the v_{adv} -wake
density	final-step	12	49.3%	time-avg far better
density	2D (y, z)	6	20.6%	best density result
particles	3D comoving	11	36.1%	harder than density
particles	2D (y, z)	14	41.4%	worse (x is signal)
<i>negative results (do not help):</i> weld-pitch inputs, log inputs, first-stage ROM coeffs, richer regressors.				

Table 4: All main variants (rbf regressor, 20 cases). LOOCV = mean held-out relative L^2 error.

8 Conclusions and next steps

1. **Feasible, not yet accurate.** Linear POD + simple regression gives ~ 21 – 28% held-out density error from two inputs — promising for a first pass, not production-grade.
2. **The bottleneck is data, not representation.** Projection error ($\sim 12\%$) $\ll$ LOOCV ($\sim 28\%$); no input encoding, normalization, or regressor helps; the active subspace is v_{adv} -dominated and low-rank. The limit is 20 samples in an effectively-1D input space.
3. **Representation matters.** Use the *time-averaged* density (not a final snapshot), and the $2D$ (y, z) *cross-section* as the density ROM target (LOOCV $\sim 21\%$, ~ 6 modes, intrinsic dim ~ 1.7). Keep full 3D for particles.
4. **The manifold is curved but smooth.** A nonlinear reduction (autoencoder / kernel-POD) is the *eventual* route to lower error, but at 20 samples it would overfit; the target latent dimension is ~ 2 – 3 .

Recommended next step. Add ~ 5 – 8 samples in the **high- $|\omega_{\text{pin}}|$ /low- v_{adv} corner** and along the convex-hull edges, where the LOOCV error concentrates. Adding the previously-missing case (which sits in that corner) already reduced the worst-case density error from $\sim 48\%$ to $\sim 43\%$ — direct empirical support for this recommendation.

Reproducibility

All code is in the `jaxtrace/rom/` package (branch `feature/rom-svd`), with drivers `run_rom_pca.py`, `run_rom_loocv.py`, `run_rom_particles.py`, `run_rom_first_stage.py`, `run_rom_manifold.py`, and `run_rom_density_compare.py`. Density product and 2D projection are selected via `--density-filename` and `--project-2d sum`; particle 2D via `--components y z`. The companion Markdown report `docs/rom_feasibility_report.md` contains the full step-by-step derivations and the complete figure set.